

Assignment 05: Stacks, Queues, and Deques

Tyler Rex | CS 210 | San Diego State University | C++17

Overview

This program implements six integer abstract data types from scratch: an array stack, linked stack, array queue, linked queue, array deque, and linked deque. It uses no `<stack>`, `<queue>`, `<deque>`, or container that implements these structures. The three array versions have a fixed capacity of five so full behavior can be demonstrated.

Compile in the Assignment05 directory with `clang++ -std=c++17 -Wall -Wextra main.cpp -o assignment05`, then run `./assignment05`. The requested build completed with no warnings. A separate AddressSanitizer and UndefinedBehaviorSanitizer run also finished with zero failures.

Design and empty handling

Stacks

The array stack stores its bottom-to-top values in `data[0]` through `data[count - 1]`. `push` writes at `count` and increments it; `pop` decrements `count` and returns that element. The linked stack keeps the top at `head`. `push` adds a node before `head`, and `pop` removes `head`. Both therefore return the newest value first (LIFO).

Queues

The array queue uses `front` and `count` as circular-array state. `enqueue` writes to `(front + count) % CAPACITY`. `dequeue` reads `front` and advances it with modulo; no shifting is needed. The linked queue keeps `head` for the oldest item and `tail` for the newest. `enqueue` links after `tail`, and `dequeue` removes `head`. When the last node is removed, `tail` becomes `nullptr`. Both preserve FIFO order.

Dequeues

The array deque uses the same circular indexing. `pushFront` moves `front` backward before writing; `pushBack` writes at `(front + count) % CAPACITY`. `popFront` advances `front`; `popBack` reads the last occupied index and decrements `count`. The linked deque is doubly linked: `head` and `tail` are the two ends, and each node has `previous` and `next` pointers. This allows all four end operations without traversal.

All removal functions return `false` on an empty structure and leave the output parameter unchanged. Full fixed arrays reject insertion and leave their contents unchanged. Linked classes delete remaining nodes in their destructors and disable copying to avoid double deletion from shallow pointer copies.

Time and space complexity

Implementation	Insertion	Removal	Check / size	Space
Array stack	$O(1)$	$O(1)$	$O(1)$	$O(C)$
Linked stack	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Array queue	$O(1)$	$O(1)$	$O(1)$	$O(C)$
Linked queue	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Array deque	$O(1)$ both ends	$O(1)$ both ends	$O(1)$	$O(C)$
Linked deque	$O(1)$ both ends	$O(1)$ both ends	$O(1)$	$O(n)$

Here n is the current item count and $C = 5$ is the fixed array capacity. The operations in this particular design never become $O(n)$: a full array insertion returns false, circular indices prevent shifting, and the linked deque has previous pointers. A resizable array would have an $O(n)$ growth step; a shifting array queue or a singly linked deque popBack could take $O(n)$. Those are comparison designs, not operations used here. Destruction of n remaining linked nodes is $O(n)$. Visiting or printing every stored item would also be $O(n)$.

Tests and actual results

The driver checks each attempted insertion or removal against an expected success result and, for successful removals, an expected value. It also checks that failed removals leave the output value unchanged. It tests empty removals before and after use, array overflow, LIFO order, FIFO order, both deque ends, circular wraparound, and an empty final size. Every structure has at least 16 operations: array stack 17, linked stack 16, array queue 17, linked queue 16, array deque 19, and linked deque 16. The actual run recorded 101 operations and 0 failures. The complete captured console output follows.

```
ARRAY STACK
1. pop -> empty [PASS]
2. push 10 -> success [PASS]
3. push 20 -> success [PASS]
4. push 30 -> success [PASS]
5. pop -> 30 [PASS]
6. push 40 -> success [PASS]
7. push 50 -> success [PASS]
8. push 60 -> success [PASS]
9. push 99 while full -> rejected [PASS]
10. pop -> 60 [PASS]
11. pop -> 50 [PASS]
12. push 70 -> success [PASS]
13. pop -> 70 [PASS]
14. pop -> 40 [PASS]
15. pop -> 20 [PASS]
16. pop -> 10 [PASS]
17. pop -> empty [PASS]
Operations: 17; final size: 0
```

```
LINKED STACK
1. pop -> empty [PASS]
2. push 10 -> success [PASS]
3. push 20 -> success [PASS]
4. push 30 -> success [PASS]
5. pop -> 30 [PASS]
6. push 40 -> success [PASS]
7. push 50 -> success [PASS]
8. push 60 -> success [PASS]
9. pop -> 60 [PASS]
10. pop -> 50 [PASS]
11. push 70 -> success [PASS]
12. pop -> 70 [PASS]
13. pop -> 40 [PASS]
14. pop -> 20 [PASS]
15. pop -> 10 [PASS]
16. pop -> empty [PASS]
Operations: 16; final size: 0
```

```
ARRAY QUEUE
1. dequeue -> empty [PASS]
2. enqueue 10 -> success [PASS]
3. enqueue 20 -> success [PASS]
4. enqueue 30 -> success [PASS]
5. enqueue 40 -> success [PASS]
6. enqueue 50 -> success [PASS]
7. enqueue 99 while full -> rejected [PASS]
8. dequeue -> 10 [PASS]
9. dequeue -> 20 [PASS]
10. enqueue 60 -> success [PASS]
11. enqueue 70 -> success [PASS]
12. dequeue -> 30 [PASS]
13. dequeue -> 40 [PASS]
14. dequeue -> 50 [PASS]
15. dequeue -> 60 [PASS]
16. dequeue -> 70 [PASS]
17. dequeue -> empty [PASS]
Operations: 17; final size: 0
```

```
LINKED QUEUE
1. dequeue -> empty [PASS]
2. enqueue 10 -> success [PASS]
3. enqueue 20 -> success [PASS]
4. enqueue 30 -> success [PASS]
5. enqueue 40 -> success [PASS]
```

```

6. enqueue 50 -> success [PASS]
7. dequeue -> 10 [PASS]

8. dequeue -> 20 [PASS]
9. enqueue 60 -> success [PASS]
10. enqueue 70 -> success [PASS]
11. dequeue -> 30 [PASS]
12. dequeue -> 40 [PASS]
13. dequeue -> 50 [PASS]
14. dequeue -> 60 [PASS]
15. dequeue -> 70 [PASS]
16. dequeue -> empty [PASS]
Operations: 16; final size: 0

ARRAY DEQUE
1. popFront -> empty [PASS]
2. pushBack 10 -> success [PASS]
3. pushFront 20 -> success [PASS]
4. pushBack 30 -> success [PASS]
5. popBack -> 30 [PASS]
6. pushFront 40 -> success [PASS]
7. pushBack 50 -> success [PASS]
8. popFront -> 40 [PASS]
9. pushBack 60 -> success [PASS]
10. popBack -> 60 [PASS]
11. pushFront 70 -> success [PASS]
12. pushBack 80 -> success [PASS]
13. pushFront 99 while full -> rejected [PASS]
14. popBack -> 80 [PASS]
15. popFront -> 70 [PASS]
16. popFront -> 20 [PASS]
17. popBack -> 50 [PASS]
18. popBack -> 10 [PASS]
19. popBack -> empty [PASS]
Operations: 19; final size: 0

LINKED DEQUE
1. popFront -> empty [PASS]
2. pushBack 10 -> success [PASS]
3. pushFront 20 -> success [PASS]
4. pushBack 30 -> success [PASS]
5. popBack -> 30 [PASS]
6. pushFront 40 -> success [PASS]
7. pushBack 50 -> success [PASS]
8. popFront -> 40 [PASS]
9. pushBack 60 -> success [PASS]
10. popBack -> 60 [PASS]
11. pushFront 70 -> success [PASS]
12. popFront -> 70 [PASS]
13. popFront -> 20 [PASS]
14. popBack -> 50 [PASS]
15. popBack -> 10 [PASS]
16. popBack -> empty [PASS]
Operations: 16; final size: 0

TOTAL: 101 operations; 0 failures

```

Complete course-relevant source code: main.cpp

The following is the complete C++17 source used to compile the program and produce the output above.

```
#include <iostream>
#include <string>

using namespace std;

// Array structures have a fixed capacity. A failed insertion changes nothing.
class ArrayStack {
private:
    static const int CAPACITY = 5;
    int data[CAPACITY];
    int count;

public:
    ArrayStack() : count(0) {}

    bool push(int value) {
        if (count == CAPACITY) return false;
        data[count++] = value;
        return true;
    }

    bool pop(int& value) {
        if (isEmpty()) return false;
        value = data[--count];
        return true;
    }

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }
};

class LinkedStack {
private:
    struct Node { int value; Node* next; };
    Node* head;
    int count;

public:
    LinkedStack() : head(nullptr), count(0) {}
    LinkedStack(const LinkedStack&) = delete;
    LinkedStack& operator=(const LinkedStack&) = delete;

    bool push(int value) {
        head = new Node{value, head};
        ++count;
        return true;
    }

    bool pop(int& value) {
        if (isEmpty()) return false;
        Node* old = head;
        value = old->value;
        head = old->next;
        delete old;
        --count;
        return true;
    }

    bool isEmpty() const { return head == nullptr; }
    int size() const { return count; }

    ~LinkedStack() {
        while (head != nullptr) {
            Node* old = head;
            head = head->next;
            delete old;
        }
    }
};

class ArrayQueue {
private:
    static const int CAPACITY = 5;
    int data[CAPACITY];
    int front;
    int count;

public:
    ArrayQueue() : front(0), count(0) {}

    bool enqueue(int value) {
        if (count == CAPACITY) return false;
```

```

        data[(front + count) % CAPACITY] = value;
        ++count;
        return true;
    }

    bool dequeue(int& value) {
        if (isEmpty()) return false;
        value = data[front];
        front = (front + 1) % CAPACITY;
        --count;
        return true;
    }

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }
};

class LinkedQueue {
private:
    struct Node { int value; Node* next; };
    Node* head;
    Node* tail;
    int count;

public:
    LinkedQueue() : head(nullptr), tail(nullptr), count(0) {}
    LinkedQueue(const LinkedQueue&) = delete;
    LinkedQueue& operator=(const LinkedQueue&) = delete;

    bool enqueue(int value) {
        Node* node = new Node{value, nullptr};
        if (tail == nullptr) head = node;
        else tail->next = node;
        tail = node;
        ++count;
        return true;
    }

    bool dequeue(int& value) {
        if (isEmpty()) return false;
        Node* old = head;
        value = old->value;
        head = old->next;
        if (head == nullptr) tail = nullptr;
        delete old;
        --count;
        return true;
    }

    bool isEmpty() const { return head == nullptr; }
    int size() const { return count; }

    ~LinkedQueue() {
        while (head != nullptr) {
            Node* old = head;
            head = head->next;
            delete old;
        }
    }
};

class ArrayDeque {
private:
    static const int CAPACITY = 5;
    int data[CAPACITY];
    int front;
    int count;

public:
    ArrayDeque() : front(0), count(0) {}

    bool pushFront(int value) {
        if (count == CAPACITY) return false;
        front = (front + CAPACITY - 1) % CAPACITY;
        data[front] = value;
        ++count;
        return true;
    }

    bool pushBack(int value) {
        if (count == CAPACITY) return false;
        data[(front + count) % CAPACITY] = value;
        ++count;
        return true;
    }

    bool popFront(int& value) {
        if (isEmpty()) return false;

```

```

        value = data[front];
        front = (front + 1) % CAPACITY;
        --count;
        return true;
    }

    bool popBack(int& value) {
        if (isEmpty()) return false;
        value = data[(front + count - 1) % CAPACITY];
        --count;
        return true;
    }

    bool isEmpty() const { return count == 0; }
    int size() const { return count; }
};

class LinkedDeque {
private:
    struct Node { int value; Node* previous; Node* next; };
    Node* head;
    Node* tail;
    int count;

public:
    LinkedDeque() : head(nullptr), tail(nullptr), count(0) {}
    LinkedDeque(const LinkedDeque&) = delete;
    LinkedDeque& operator=(const LinkedDeque&) = delete;

    bool pushFront(int value) {
        Node* node = new Node{value, nullptr, head};
        if (head == nullptr) tail = node;
        else head->previous = node;
        head = node;
        ++count;
        return true;
    }

    bool pushBack(int value) {
        Node* node = new Node{value, tail, nullptr};
        if (tail == nullptr) head = node;
        else tail->next = node;
        tail = node;
        ++count;
        return true;
    }

    bool popFront(int& value) {
        if (isEmpty()) return false;
        Node* old = head;
        value = old->value;
        head = old->next;
        if (head == nullptr) tail = nullptr;
        else head->previous = nullptr;
        delete old;
        --count;
        return true;
    }

    bool popBack(int& value) {
        if (isEmpty()) return false;
        Node* old = tail;
        value = old->value;
        tail = old->previous;
        if (tail == nullptr) head = nullptr;
        else tail->next = nullptr;
        delete old;
        --count;
        return true;
    }

    bool isEmpty() const { return head == nullptr; }
    int size() const { return count; }

    ~LinkedDeque() {
        while (head != nullptr) {
            Node* old = head;
            head = head->next;
            delete old;
        }
    }
};

void checkInsert(const string& action, bool actual, bool expected,
                int& steps, int& failures) {
    ++steps;
    bool pass = actual == expected;
    if (!pass) ++failures;
    cout << steps << ". " << action << " -> "

```

```

        << (actual ? "success" : "rejected")
        << " [" << (pass ? "PASS" : "FAIL") << "]\n";
    }

void checkRemove(const string& action, bool actual, int value,
                bool expected, int expectedValue, int& steps, int& failures) {
    ++steps;
    bool pass = actual == expected && value == expectedValue;
    if (!pass) ++failures;
    cout << steps << ". " << action << " -> ";
    if (actual) cout << value;
    else cout << "empty";
    cout << " [" << (pass ? "PASS" : "FAIL") << "]\n";
}

template <typename Stack>
void testStack(const string& name, Stack& stack, bool bounded,
              int& totalSteps, int& failures) {
    int steps = 0;
    int value = -1;
    auto pop = [&](bool expected, int expectedValue = 0) {
        int before = value;
        bool actual = stack.pop(value);
        checkRemove("pop", actual, value, expected,
                    expected ? expectedValue : before, steps, failures);
    };
    cout << "\n" << name << "\n";
    pop(false);
    checkInsert("push 10", stack.push(10), true, steps, failures);
    checkInsert("push 20", stack.push(20), true, steps, failures);
    checkInsert("push 30", stack.push(30), true, steps, failures);
    pop(true, 30);
    checkInsert("push 40", stack.push(40), true, steps, failures);
    checkInsert("push 50", stack.push(50), true, steps, failures);
    checkInsert("push 60", stack.push(60), true, steps, failures);
    if (bounded)
        checkInsert("push 99 while full", stack.push(99), false, steps, failures);
    pop(true, 60);
    pop(true, 50);
    checkInsert("push 70", stack.push(70), true, steps, failures);
    pop(true, 70);
    pop(true, 40);
    pop(true, 20);
    pop(true, 10);
    pop(false);
    if (!stack.isEmpty() || stack.size() != 0) ++failures;
    cout << "Operations: " << steps << "; final size: " << stack.size() << "\n";
    totalSteps += steps;
}

template <typename Queue>
void testQueue(const string& name, Queue& queue, bool bounded,
              int& totalSteps, int& failures) {
    int steps = 0;
    int value = -1;
    auto dequeue = [&](bool expected, int expectedValue = 0) {
        int before = value;
        bool actual = queue.dequeue(value);
        checkRemove("dequeue", actual, value, expected,
                    expected ? expectedValue : before, steps, failures);
    };
    cout << "\n" << name << "\n";
    dequeue(false);
    checkInsert("enqueue 10", queue.enqueue(10), true, steps, failures);
    checkInsert("enqueue 20", queue.enqueue(20), true, steps, failures);

    checkInsert("enqueue 30", queue.enqueue(30), true, steps, failures);
    checkInsert("enqueue 40", queue.enqueue(40), true, steps, failures);
    checkInsert("enqueue 50", queue.enqueue(50), true, steps, failures);
    if (bounded)
        checkInsert("enqueue 99 while full", queue.enqueue(99), false, steps, failures);
    dequeue(true, 10);
    dequeue(true, 20);
    checkInsert("enqueue 60", queue.enqueue(60), true, steps, failures);
    checkInsert("enqueue 70", queue.enqueue(70), true, steps, failures);
    dequeue(true, 30);
    dequeue(true, 40);
    dequeue(true, 50);
    dequeue(true, 60);
    dequeue(true, 70);
    dequeue(false);
    if (!queue.isEmpty() || queue.size() != 0) ++failures;
    cout << "Operations: " << steps << "; final size: " << queue.size() << "\n";
    totalSteps += steps;
}

template <typename Deque>
void testDeque(const string& name, Deque& deque, bool bounded,
              int& totalSteps, int& failures) {

```

```

int steps = 0;
int value = -1;
auto popFront = [&](bool expected, int expectedValue = 0) {
    int before = value;
    bool actual = deque.popFront(value);
    checkRemove("popFront", actual, value, expected,
        expected ? expectedValue : before, steps, failures);
};
auto popBack = [&](bool expected, int expectedValue = 0) {
    int before = value;
    bool actual = deque.popBack(value);
    checkRemove("popBack", actual, value, expected,
        expected ? expectedValue : before, steps, failures);
};
cout << "\n" << name << "\n";
popFront(false);
checkInsert("pushBack 10", deque.pushBack(10), true, steps, failures);
checkInsert("pushFront 20", deque.pushFront(20), true, steps, failures);
checkInsert("pushBack 30", deque.pushBack(30), true, steps, failures);
popBack(true, 30);
checkInsert("pushFront 40", deque.pushFront(40), true, steps, failures);
checkInsert("pushBack 50", deque.pushBack(50), true, steps, failures);
popFront(true, 40);
checkInsert("pushBack 60", deque.pushBack(60), true, steps, failures);
popBack(true, 60);
checkInsert("pushFront 70", deque.pushFront(70), true, steps, failures);
if (bounded) {
    checkInsert("pushBack 80", deque.pushBack(80), true, steps, failures);
    checkInsert("pushFront 99 while full", deque.pushFront(99), false, steps, failures);
    popBack(true, 80);
}
popFront(true, 70);
popFront(true, 20);
popBack(true, 50);
popBack(true, 10);
popBack(false);
if (!deque.isEmpty() || deque.size() != 0) ++failures;
cout << "Operations: " << steps << "; final size: " << deque.size() << "\n";
totalSteps += steps;
}

int main() {
    int totalSteps = 0;
    int failures = 0;
    ArrayStack arrayStack;
    LinkedStack linkedStack;
    ArrayQueue arrayQueue;
    LinkedQueue linkedQueue;
    ArrayDeque arrayDeque;
    LinkedDeque linkedDeque;

    testStack("ARRAY STACK", arrayStack, true, totalSteps, failures);
    testStack("LINKED STACK", linkedStack, false, totalSteps, failures);
    testQueue("ARRAY QUEUE", arrayQueue, true, totalSteps, failures);
    testQueue("LINKED QUEUE", linkedQueue, false, totalSteps, failures);
    testDeque("ARRAY DEQUE", arrayDeque, true, totalSteps, failures);
    testDeque("LINKED DEQUE", linkedDeque, false, totalSteps, failures);

    cout << "\nTOTAL: " << totalSteps << " operations; "
        << failures << " failures\n";
    return failures == 0 ? 0 : 1;
}

```